

Day07-SELinux与存储管理

课程目标

- 理解 SELinux 基本概念
- 掌握 SELinux 模式切换
- 学会管理 SELinux 文件上下文
- 掌握 SELinux 布尔值配置
- 理解磁盘分区和文件系统
- 掌握交换空间管理

上午 - SELinux安全管理

课前故事：SELinux 的诞生

为什么需要 SELinux?

传统 Linux 安全 (DAC) 的局限性:

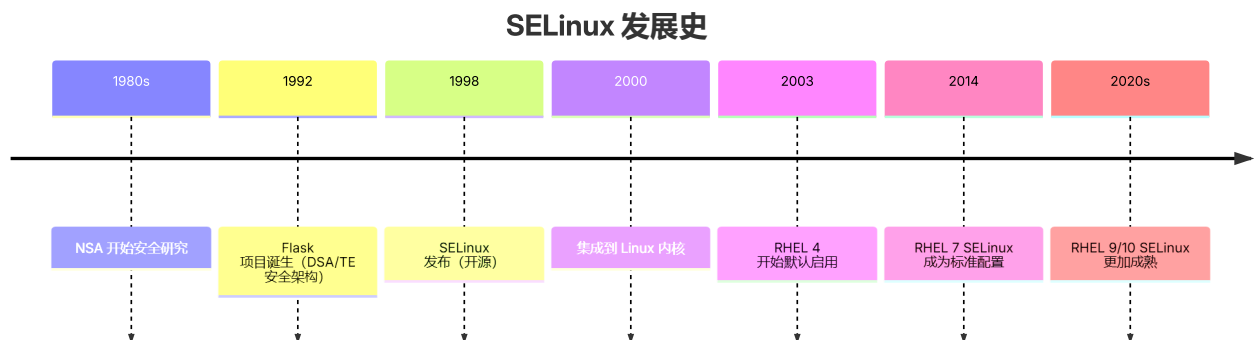
大多数 Linux 管理员都熟悉标准 user/group/other 权限安全模型，SELinux 提供另一层安全，它基于对象并由更加复杂的规则控制，称为强制访问控制，SELinux(Security-Enhanced Linux) 是美国国家安全局(NSA)对于强制访问控制的实现。

并非所有安全问题都可以提前预测。SELinux 可以防止因为应用程序的漏洞而影响其他应用访问。例如：要允许远程匿名访问 Web 服务器，必须打开防火墙端口。这让恶意人员有机会通过安全漏洞侵入系统。如果成功入侵 Web 服务器进程，就可以获得 apache 用户和 apache 组的权限。该用户和组对文档根目录 /var/www/html 具有读取权限。它还可以访问 /tmp 和 /var/tmp，以及全局可写的任何文件和目录。



为了保护这些资源不被恶意使用，SELinux 提供一组安全规则，定义进程可以访问哪些文件、目录和端口。

SELinux 的发展历史：



SELinux 的革命性设计：

SELinux 实现多层安全防护：

1. 应用程序 提出访问请求
2. **DAC** 层 - 标准权限 (rwx)，基于用户/组控制
3. **MAC** 层 - SELinux 策略，基于 context，实现进程隔离和最小权限
4. 系统资源 - 最终被访问的目标

第一部分：SELinux基础

本部分将学习：

- SELinux 的基本概念和工作原理
- 三种运行模式及其切换方法
- 文件上下文的管理和恢复
- 布尔值配置来调整策略行为

掌握这些知识，你将能够：

- 快速诊断 SELinux 相关的服务访问问题
- 正确配置服务以符合 SELinux 策略
- 在考试中处理 SELinux 相关的题目

1.1 SELinux 介绍

什么是 **SELinux**：

SELinux (Security-Enhanced Linux) 是强制访问控制系统

- 提供 DAC 之外的安全层
- 基于标签的安全策略
- 保护系统免受应用程序漏洞影响
- NSA 开发的安全机制

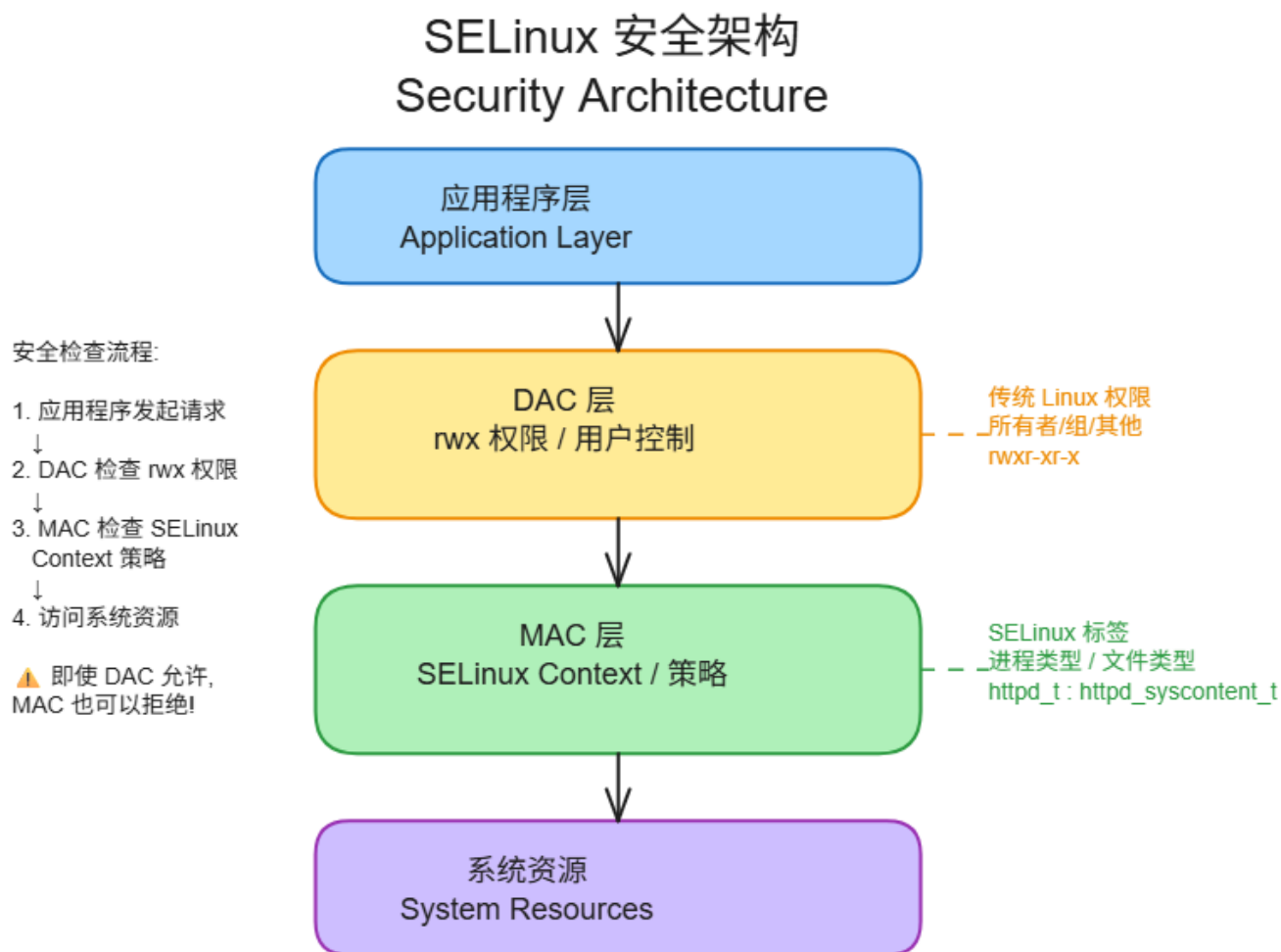
DAC vs MAC：

特性	DAC (自主访问控制)	MAC (强制访问控制)
基于权限	rwx 权限	SELinux context
用户控制	文件所有者	系统策略
粒度	粗粒度	细粒度
保护对象	文件/目录	文件/目录/端口/进程

SELinux 保护的详细原理：

SELinux 提供一组安全规则，定义进程可以访问哪些文件、目录和端口：

- 每个文件、进程、目录和端口都有专门的安全标签，称为SELinux 上下文 (context)
- SELinux 策略使用 context 来确定某个进程能否访问文件、目录或端口。如果没有允许规则，则不允许访问
- SELinux 标签具有多种 context: user, role, type, 和sensitivity。
- 目标策略 (targeted policy 是 RHEL 中启用的默认策略) 会根据第三个 context (即 type context 来制定自己的规则。



1.2 SELinux 模式

三种模式:

模式	说明
Enforcing	强制执行策略, 拒绝违规访问
Permissive	记录违规但不拒绝, 用于测试
Disabled	完全禁用 SELinux

查看当前模式：

```
getenforce
```

SHELL

```
# 输出示例：  
# Enforcing
```

临时切换模式：

```
# 切换到 Permissive 模式  
setenforce 0
```

SHELL

```
# 切换到 Enforcing 模式  
setenforce 1
```

```
# 验证  
getenforce
```

⚠ 易错点：临时 vs 永久修改

setenforce 与 配置文件的区别：

方法	命令	效果	持久性
临时	setenforce 0 或 setenforce 1	立即生效	重启后恢复
永久	编辑 /etc/selinux/config	需重启生效	持久保存

考试陷阱：

- setenforce 改动不会保存到配置文件
- 重启后 SELinux 模式会回到 /etc/selinux/config 的值
- 需要永久修改时，必须同时修改配置文件

1.3 永久配置 SELinux

配置文件：

```
# 系统在启动时读取并配置
/etc/selinux/config
```

配置项:

```
# SELinux 模式
SELINUX=enforcing

# 策略类型
SELINUXTYPE=targeted
```

注意:

- RHEL9 起不再支持 `SELINUX=disabled`
- 需要完全禁用需在内核参数添加 `selinux=0`

修改配置后重启生效

相关练习：RH134V9 教案练习

练习名称	对应章节	说明
练习：CHANGING THE SELINUX ENFORCEMENT MODE	第06章 第01节 P180	更改SELinux强制模式

第二部分：SELinux 文件上下文

所有进程和文件都有相应的 label，代表了与安全有关的信息，称为 SELinux context。

- SELinux 在 `/etc/selinux/targeted/contexts/files/` 目录中维护基于文件的文件标签策略数据库。新文件在文件名与现有标签策略匹配时获得默认标签
- 新文件通常从父目录继承其 SELinux context (`mv` 或 `cp -a` 例外)

2.1 查看文件上下文

ls -Z 显示 SELinux 上下文:

SHELL

```
ls -Z /var/www/html/

# 输出示例:
# -rw-r--r--. root root
unconfined_u:object_r:httpd_sys_content_t:s0 index.html
```

ls -Zd 或 ll -dZ 命令显示目录的 SELinux context

SHELL

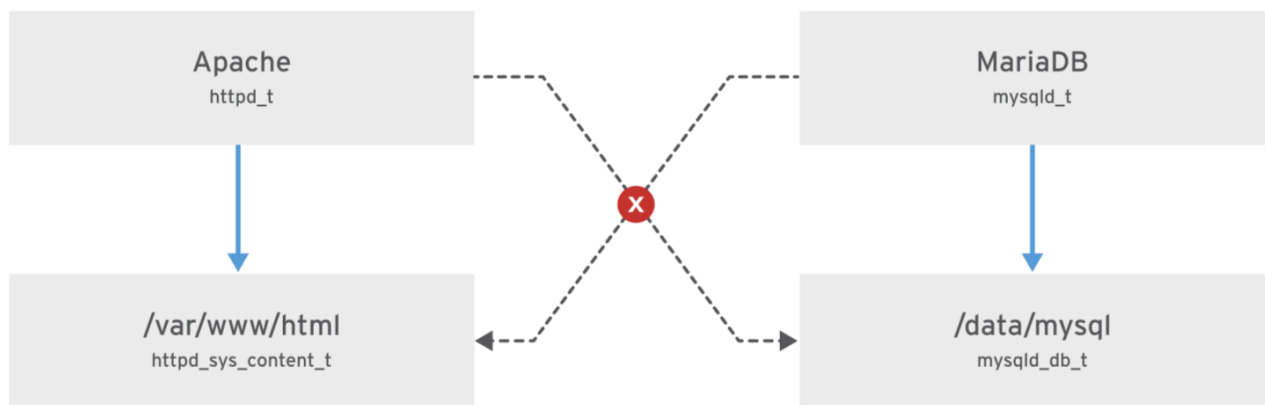
```
$ ll -dZ /var/www/html
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_content_t:s0
56 Feb  4 16:57 /var/www/html
```

上下文字段说明:

SELinux User / Role / Type / Level / File
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/file2

字段	说明	示例
user	用户身份	unconfined_u
role	角色	object_r
type	类型	httpd_sys_content_t
level	级别	s0

- Web 服务器的 type context 是 httpd_t。位于 /var/www/html 中的文件和目录的 type context 是 httpd_sys_content_t。位于 /tmp 和 /var/tmp 中的文件和目录 type context 是 tmp_t。Web 服务器端口的 type context 是 http_port_t
- 在启用 SELinux 的情况下，破坏了 Web 服务器进程的恶意用户将无法访问 /tmp 目录
- MariaDB 服务器的 type context mysqld_t。默认情况下，在 /data/mysql 中的文件具有 mysqld_db_t type context。该 type context 允许 MariaDB 访问这些文件，但禁止其他服务（如 Apache 服务）访问



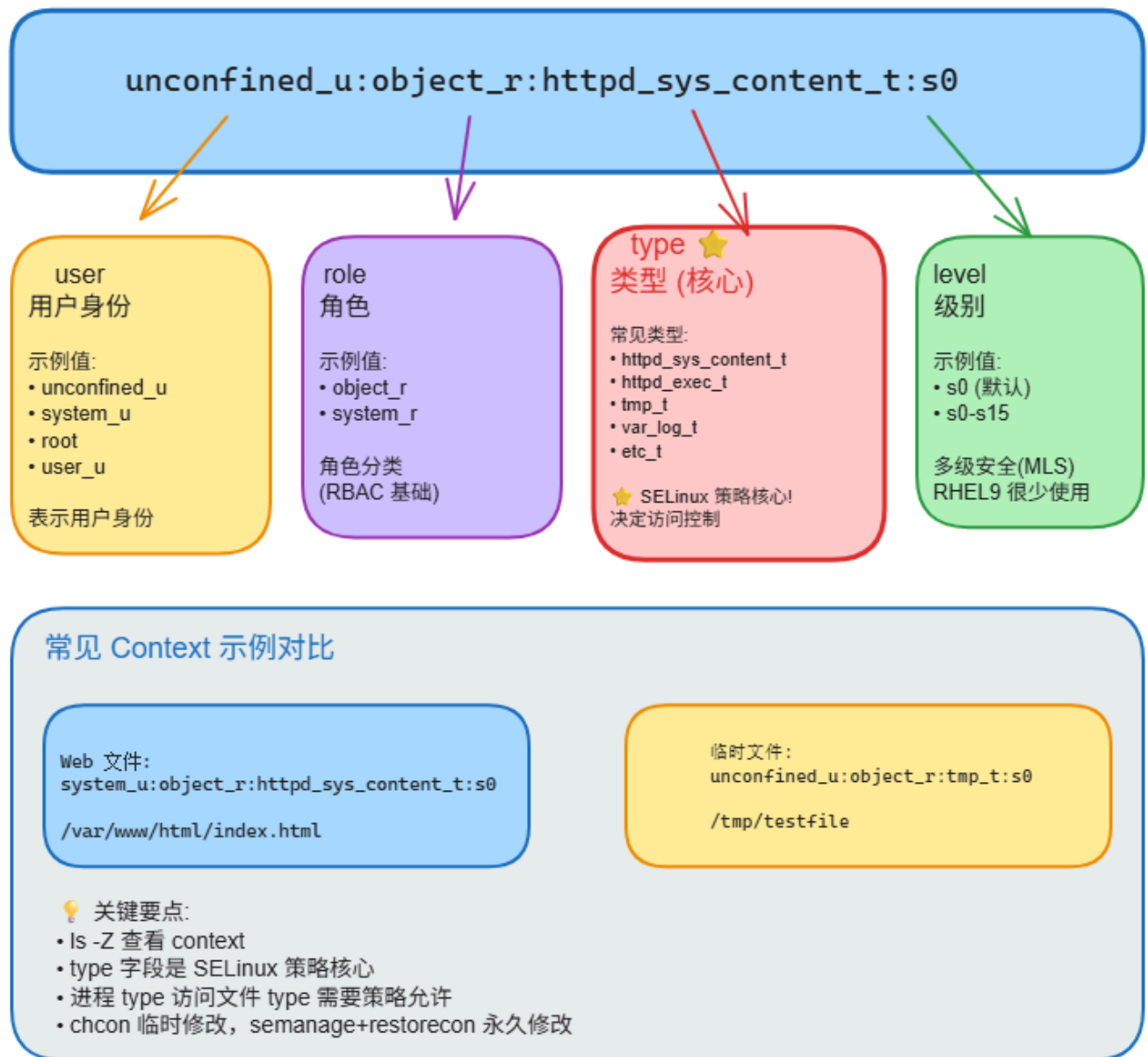
- 许多命令都使用 `-Z` 选项来显示或设置 SELinux context。例如, `ps`、`ls`、`cp` 和 `mkdir`

type context 详细说明:

服务	进程 type	文件 type	端口 type
Web 服务器	<code>httpd_t</code>	<code>httpd_sys_content_t</code>	<code>http_port_t</code>
数据库	<code>mysqld_t</code>	<code>mysqld_db_t</code>	<code>mysql_port_t</code>
FTP	<code>ftpd_t</code>	<code>public_content_t</code>	<code>ftp_port_t</code>
DNS	<code>named_t</code>	<code>named_zone_t</code>	<code>dns_port_t</code>
临时文件	-	<code>tmp_t</code>	-

SELinux Context 结构详解

Security Context Breakdown



查看目录上下文:

```
ls -Zd /var/www/html
```

SHELL

输出示例:

```
# drwxr-xr-x. root root system_u:object_r:httpd_sys_content_t:s0
/var/www/html/
```

2.2 管理文件上下文

semanage fcontext - 定义默认上下文:

SHELL

```
# 添加目录的默认上下文规则 (/.*)? 海盗
semanage fcontext -a -t httpd_sys_content_t '/virtual(/.*)?'

# 查看规则
semanage fcontext -l | grep /virtual
```

semanage fcontext 完整选项表:

选项	说明
<code>-a, --add</code>	添加指定对象类型的记录
<code>-m, --modify</code>	修改指定对象类型的记录
<code>-d, --delete</code>	删除指定对象类型的记录
<code>-l, --list</code>	列出指定对象类型的记录
<code>-t, --type</code>	指定 SELinux 类型

正则表达式扩展说明:

SHELL

```
# (/path/.*)? 的含义:
# (/path/.*)? = 匹配以 /path 开头的路径
# ( ... )?    = 此路径部分是可选的
# .*          = 任意数量的斜杠和任意字符

# 示例:
semanage fcontext -a -t httpd_sys_content_t '/virtual(/.*)?'
# 匹配 /virtual 及其下的所有内容
```

restorecon - 应用上下文:

```
# 恢复文件上下文
restorecon -Rv /virtual

# 输出示例:
# Relabeled /virtual/index.html from
unconfined_u:object_r:default_t:s0 to
system_u:object_r:httpd_sys_content_t:s0
```

参数说明:

- `-R` = 递归
- `-v` = 显示详细信息

chcon vs semanage+restorecon:

方法	命令	持久性	推荐度
临时修改	<code>chcon -t type file</code>	否 (运行 restorecon 后还原)	 不推荐
永久修改	<code>semanage fcontext -a -t type '/path' + restorecon -Rv /path</code>	是	 推荐

为什么使用 semanage+restorecon:

```
# chcon 只是临时修改, 不会保存到策略数据库
chcon -t httpd_sys_content_t /var/www/html/file1

# semanage 修改策略数据库, restorecon 应用策略
semanage fcontext -a -t httpd_sys_content_t '/var/www/html/file1'
restorecon -Rv /var/www/html
```

 易错点: chcon vs semanage

chcon 与 semanage+restorecon 对比:

方法	命令	持久性	特点
chcon (临时)	<code>chcon -t httpd_sys_content_t /web/file.html</code>	✗ 临时	restorecon 后会还原
semanage+restorecon (永久)	<code>semanage fcontext -a -t httpd_sys_content_t '/web' restorecon -R -v /web</code>	✓ 永久	写入策略数据库

相关练习：RH134V9 教案练习

练习名称	对应章节	说明
练习：CONTROLLING SELINUX FILE CONTEXTS	第06章 第02节 P188	控制SELinux文件上下文

第三部分：SELinux 布尔值

应用或服务开发人员编写 SELinux 目标策略来定义目标应用的允许行为。开发人员可以在 SELinux策略中包含可选的应用行为，当特定系统上允许相关行为时启用该策略。SELinux布尔值可启用或禁用SELinux策略的可选行为。通过使用布尔值，您可以有选择地调整应用的行为。

这些可选行为是特定于应用的，必须为各个目标应用发现和选择。如需了解服务相关的布尔值，请参阅该服务的 SELinux man page。例如，Web 服务器 httpd 服务有其 httpd(8) man page，以及用于记录其 SELinux 策略的 httpd_selinux(8) man page，包括支持的进程类型、文件上下文和可用的布尔值行为。SELinux man page 在 se linux-policy-doc 软件包中提供。

3.1 布尔值管理

使用getsebool命令列出此系统上目标策略的可用布尔值，以及当前的布尔值状态。

```
# 列出所有布尔值
getsebool -a

# 查看特定布尔值
getsebool httpd_enable_homedirs

# 输出示例:
# httpd_enable_homedirs --> off
```

semanage boolean - 查看详细信息:

```
# 查看布尔值和描述
semanage boolean -l | grep httpd

# 输出示例:
# httpd_enable_homedirs (off , off) Allow httpd to enable
homedirs
```

使用 **sesearch** 搜索策略规则

```
# 安装工具包 (如果没有)
yum install setools-console -y

# 查看布尔值影响的规则
sesearch -b httpd_enable_homedirs -A

# 输出示例:
# allow httpd_user_content_t httpd_sys_script_exec_t:file {
execute };
# allow httpd_user_content_t httpd_t:file { read };
```

设置布尔值:

使用setsebool命令启用或禁用这些行为的运行状态。 **setsebool -P**命令选项通过写入策略文件使设置持久有效。只有特权用户才能设置SELinux布尔值。

```
# 临时设置(重启后失效)
setsebool httpd_enable_homedirs on

# 永久设置
setsebool -P httpd_enable_homedirs on

# 验证
getsebool httpd_enable_homedirs
```

参数说明：

- **-P** = permanent 永久生效

⚠ 易错点：布尔值的 **-P** 选项

setsebool 的 -P 选项：

方法	命令	持久性	说明
临时设置	<code>setsebool httpd_enable_homedirs on</code>	✗ 重启后失效	仅适合测试
永久设置	<code>setsebool -P httpd_enable_homedirs on</code>	✓ 写入配置文件	考试和生产推荐

记忆：P = Permanent (永久)

查看修改过的布尔值：

```
semanage boolean -l -C

# 输出说明：
# 显示当前状态与默认状态不同的布尔值
```

相关练习：RH134V9

练习名称	对应章节	说明
练习：ADJUSTING SELINUX POLICY WITH BOOLEANS	第06章 第03节 P193	调整SELinux布尔值

第四部分：SELinux 故障排除

4.1 问题诊断

常见 SELinux 问题：

- 1. 文件上下文不正确
- 2. 布尔值未启用
- 3. 服务端口未开放

诊断：

SHELL

```
# 从系统日志搜索 SELinux 关键字
grep SELinux /var/log/messages
```

4.2 使用 sealert

sealert 分析工具：

SHELL

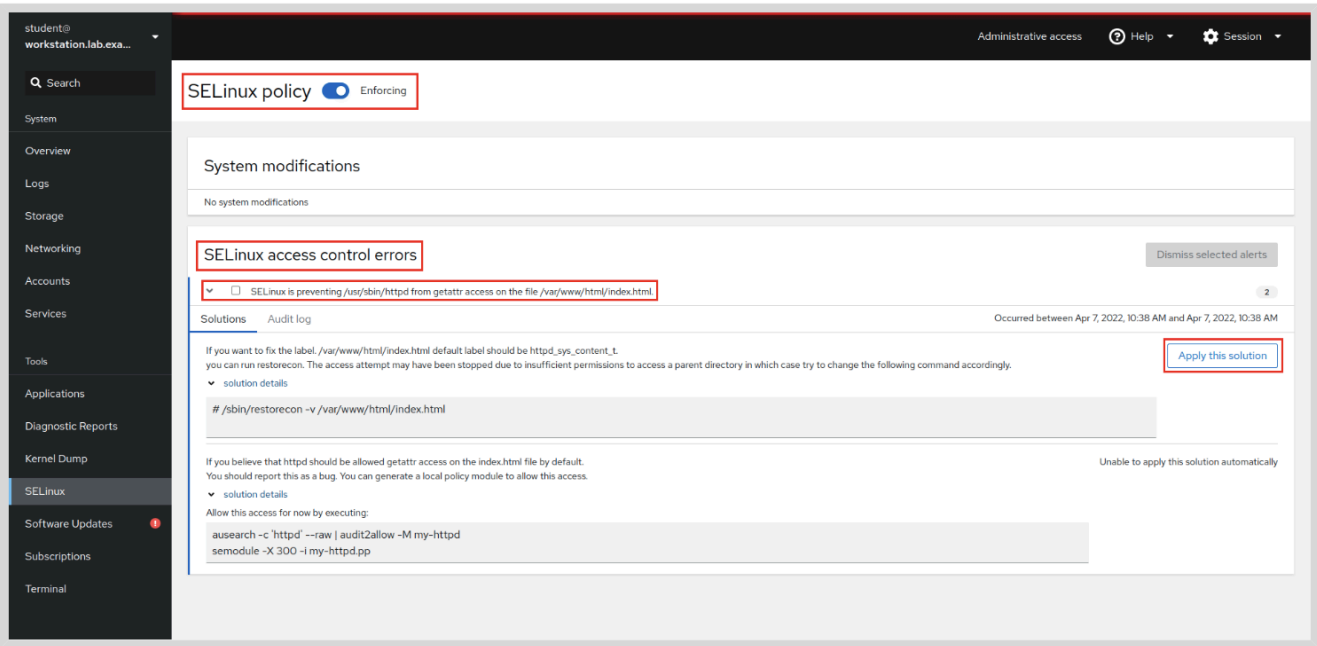
```
# 分析所有 SELinux 事件
sealert -a /var/log/audit/audit.log

# 生成报告
sealert -a /var/log/audit/audit.log > report.txt
```

4.3 使用 Web Console 进行 SELinux 问题故障排除

- 激活 web console: `systemctl enable --now cockpit.socket`

- RHEL Web Console 包含用于对 SELinux 问题进行故障排除的工具
- 左侧菜单中选择 SELinux。SELinux policy 窗口会显示当前的执行状态，SELinux access control errors 部分列出了当前的 SELinux 问题



- 单击 > 字符显示事件详细信息
- 单击 solution details 以显示所有事件详细信息和建议
- 可以单击 Apply the solution 来应用所给的建议
- 更正问题后，SELinux access control errors 部分应从视图中删除该事件。如果显示 No SELinux alerts 消息，则表示您已更正了当前的所有 SELinux 问题

相关练习：RH134V9

练习名称	对应章节	说明
练习：INVESTIGATING AND RESOLVING SELinux	第06章 第04节 P193	调查和解决SELinux问题

进阶知识：SELinux 端口管理

SELinux 通过端口上下文控制服务可以监听哪些端口。即使文件权限和防火墙都正确，如果端口不在 SELinux 策略中，服务也无法启动。

常见端口类型：

服务	端口类型	默认端口
HTTP	http_port_t	80, 443, 8008, 8009, 8443
FTP	ftp_port_t	21, 990
DNS	dns_port_t	53
MySQL	mysql_port_t	3306

semanage port 命令：

选项	说明
-a, --add	添加端口
-d, --delete	删除端口
-m, --modify	修改端口
-l, --list	列出端口规则
-t, --type	指定端口类型
-p, --proto	指定协议 (tcp/udp)

查看端口规则：

SHELL

```
# 查看所有端口规则
semanage port -l

# 查看特定类型的端口
semanage port -l | grep http_port_t

# 输出示例：
# http_port_t      tcp      80, 443, 8008, 8009, 8443
```

添加端口：

SHELL

```
# 让 httpd 监听 8888 端口
semanage port -a -t http_port_t -p tcp 8888
```

删除端口：

```
semanage port -d -t http_port_t -p tcp 8888
```

⚠ 注意：每个端口只能属于一个 SELinux 类型。如果端口已被占用，会报错 `ValueError: Port tcp/端口 already defined`。

综合练习：RH134V9

练习名称	对应章节	说明
总练习：MANAGING SELINUX SECURITY	第06章	SELinux安全管理综合练习

下午 - 存储管理

课前故事：存储技术的演进

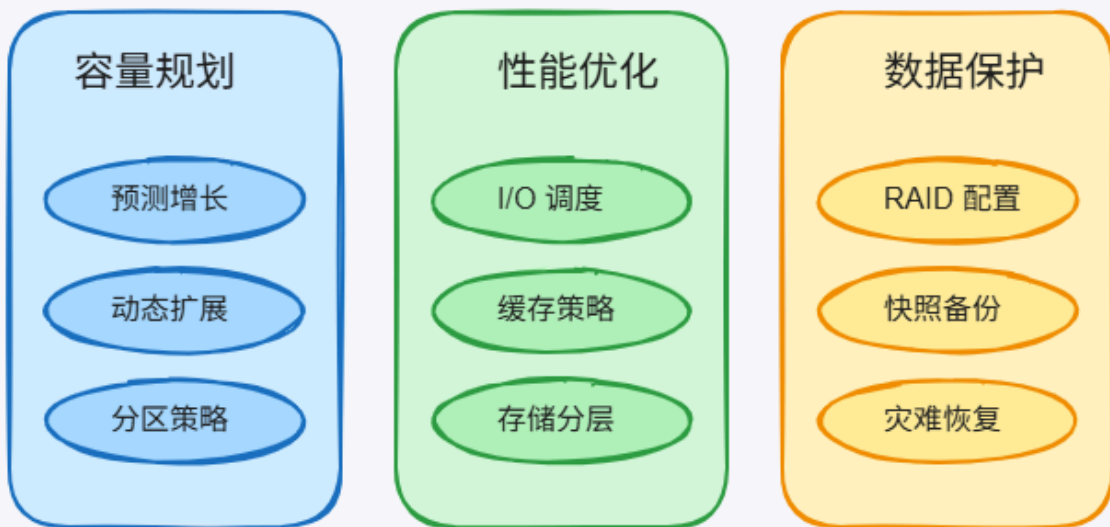
为什么需要理解存储管理？

数据是企业最宝贵的资产，而存储管理是保护数据的基础：

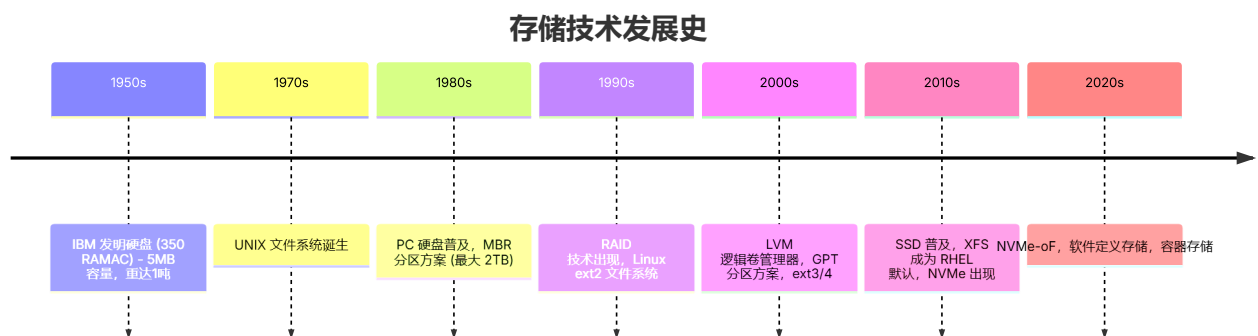
系统架构演进：

年代	架构	存储	访问方式	特点
1980年代	单机服务器	本地磁盘	直接访问	性能受限
2000年代	存储网络	SAN/NAS	网络块设备	扩展性好
2020年代	云存储/容器	分布式存储	软件定义存储	弹性伸缩

存储管理挑战



存储管理的发展历史:



本部分将学习

第五部分：磁盘分区

- MBR vs GPT 分区方案的差异
- g(d)isk, parted 工具的使用方法
- 创建、删除、管理分区

第六部分：文件系统

- XFS 和 ext4 文件系统特点

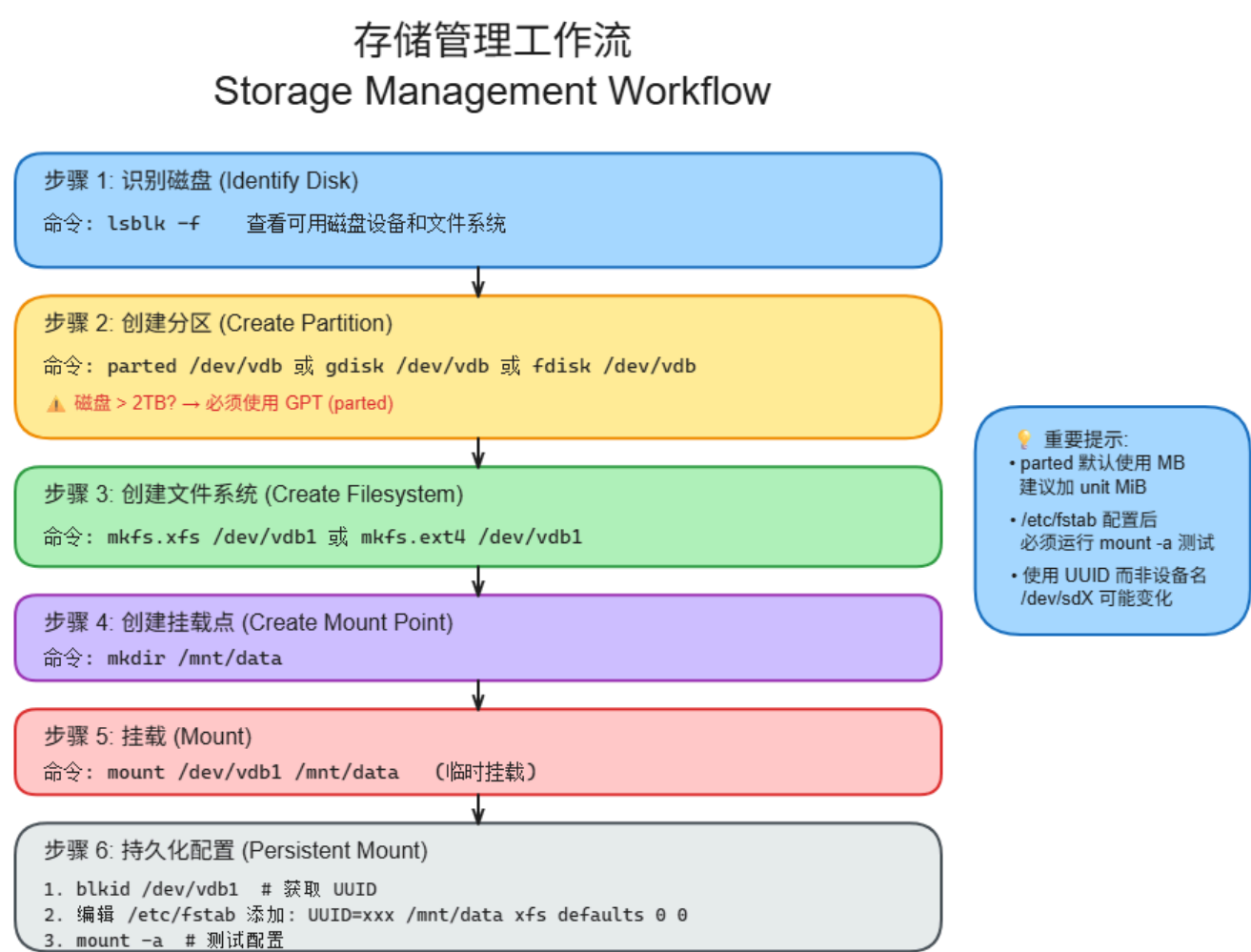
- 创建、挂载、卸载文件系统
- `/etc/fstab` 配置

第七部分：SWAP 交换空间

- SWAP 的作用和管理
- 创建 SWAP 分区和文件
- SWAP 优先级设置

第五部分：磁盘分区

存储管理工作流：



5.1 分区方案

MBR 分区方案：

- 诞生自1982年的 MBR 分区方案在 BIOS 固件的系统上最多支持4个主分区，通过扩展分区和逻辑分区，最多创建15个分区
- 分区大小数据以32位值存储，最大磁盘和分区大小为 2TiB
- MBR 存在单点故障
- 2 TiB 磁盘和分区大小限制是 MBR 局限。因此 MBR 方案已被GUID 分区表 (GPT) 分区方案取代

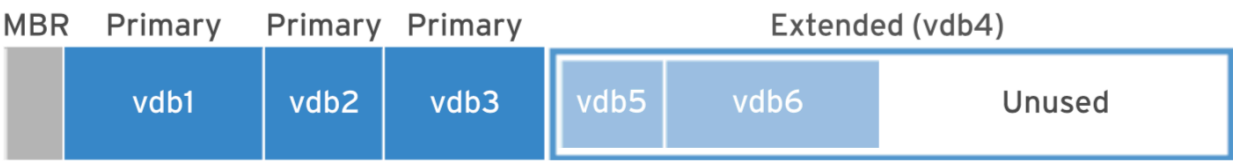


Figure 6.1: MBR Partitioning of the /dev/vdb storage device

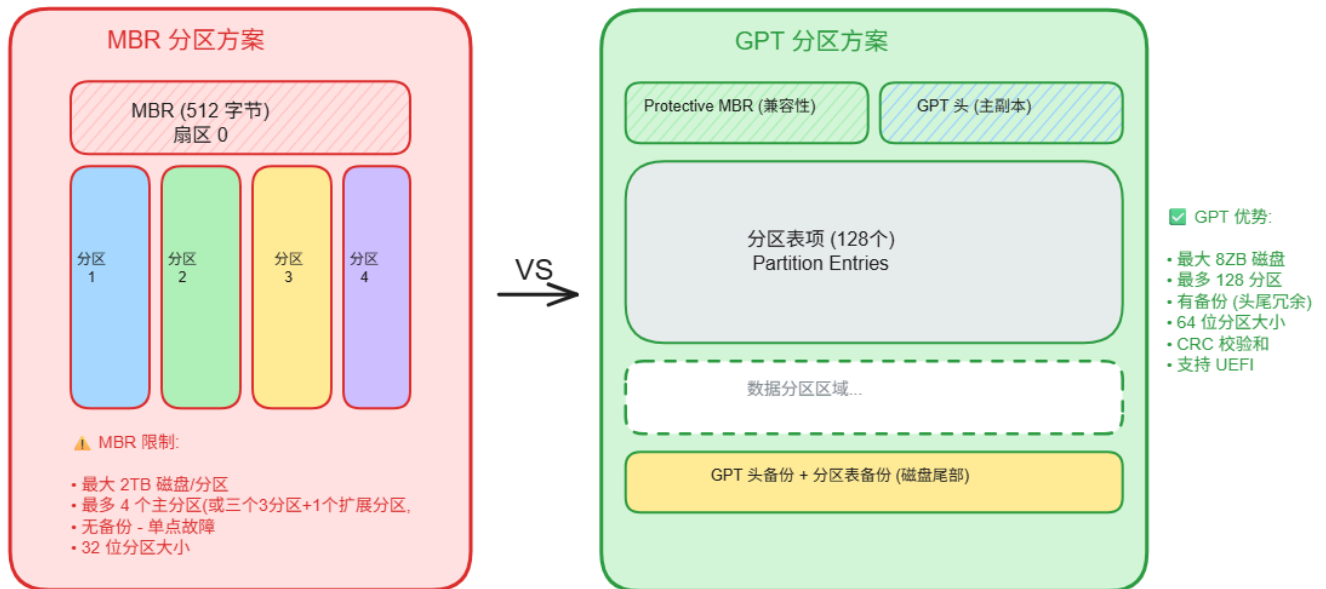
GPT 分区方案：

- 对于使用 UEFI 固件的系统，默认使用 GPT 分区方案，解决MBR 方案的限制
- GTP 最多128 分区， 64位值存储分区大小，最大磁盘和分区大小可以达到 8ZiB
- GPT 使用全局唯一标识 符 (GUID) 来识别每个磁盘和分区。GPT提供分区表信息的冗余。主 GPT 位于磁盘头部，而备份副本位于磁盘尾部。GPT 使用 checksum 来检测 GPT 头和分区表中的错误与损坏



Figure 6.2: GPT Partitioning of the /dev/vdb storage device

MBR vs GPT 分区方案对比



MBR 限制说明:

- 分区大小数据以 32 位值存储
- 2TiB 磁盘和分区大小限制
- 单点故障风险
- 已被 GPT 分区方案取代

GPT 优势说明:

- 使用 64 位值存储分区大小
- 提供分区表信息的冗余 (主副本在磁盘头部, 备份副本在尾部)
- 使用 checksum 检测损坏和错误
- 最多支持 128 个分区

f(g)disk vs parted:

工具	特点
fdisk	历史最受欢迎, 交互式操作, 可直接指定分区大小, 支持 MBR 和 GPT
gdisk	fdisk 变体, 专为 GPT 设计, fdisk的现代化替代工具
parted	同时支持 MBR 和 GPT, 交互式和非交互式

parted 命令详解:

```
# 显示磁盘分区表
parted /dev/vdb print

# 交互式访问磁盘
parted /dev/vdb
# 在交互界面中:
# - print      # 显示分区表
# - mklabel    # 创建分区表 (msdos 或 gpt),
# - mkpart     # 创建分区
# - quit       # 退出

# 更改单位
parted /dev/vdb unit MiB      # 使用二进制单位(推荐)
parted /dev/vdb unit s       # 使用扇区
parted /dev/vdb unit MB      # 使用十进制单位

# 打印分区表
parted /dev/vdb print

# 非交互式创建分区

# 创建/更改磁盘分区表类型(对新磁盘可安全执行)
# 若已存在分区会先清空分区数据必须小心操作)
parted /dev/vdb mklabel gpt

# 对于GPT分区 (第一个分区前需要预留2048扇区大小, part_name 分区名可省略)
# 2048s (1MiB) 是现代磁盘的标准对齐单位, 确保最佳 I/O 性能。
parted /dev/vdb mkpart <part_name> xfs 2048s 1001MiB

# 确保设备就绪
udevadm settle
```

parted 单位说明:

- 默认使用 10 进制单位 (KB, MB, GB)
- MiB, GiB, TiB = 二进制单位 (推荐使用)
- s = 扇区
- 计算公式: $SIZE = END - START$

parted 单位区分 - MB vs MiB:

	单位类型	符号	换算
	10进制 (默认)	MB	1000 字节
	10进制	GB	1000 ³ 字节
	2进制 (推荐)	MiB	1024×1024 字节
	2进制	GiB	1024 ³ 字节

示例差异:

- 1000MB = 953.67 MiB
- 1000MiB = 1048.58 MB
- 差异约 95 MiB!

推荐命令: parted /dev/vdb unit MiB

注意:

使用 parted 对已有MBR分区类型磁盘进行分区时, 建议设置分区名来区分主分区还是扩展分区 parted /dev/vdb mkpart primary xfs 2048s 1000MB

fdisk - 交互式分区工具:

SHELL

```
# 启动 fdisk
fdisk /dev/vdb

# 常用命令:
# o - 重建gpt分区表类型(Y确认, 会清除已有分区, 小心操作)
# m - 帮助
# p - 打印分区表
# n - 新建分区
# d - 删除分区
# t - 改变分区类型
# w - 写入并退出
# q - 退出不保存
```

分区步骤:

1. 创建分区表(会清空已有分区数据)

`fdisk /dev/vdb`

> o # 创建新的空 MBR (DOS)

或

> g # 创建新的空 GPT

2. 创建分区

> n # 新建分区

> p # 主分区

> 1 # 分区号, 从1开始, 回车使用默认值(递增1)

> # 回车使用默认起始扇区

> +512M # 大小

3. 保存退出

> w

gdisk - GPT 专用分区工具:

推荐考试时使用

启动 gdisk (仅支持 GPT)

`gdisk /dev/vdb`

o - 创建新的空GPT存储设备

p - 打印分区表

n - 新建分区

d - 删除分区

w - 写入并退出

q - 退出不保存

分区步骤:

```
# 1. 创建 GPT 分区表
gdisk /dev/vdb
# o      # 创建新的空GPT存储设备(Y确认, 会清除已有分区, 小心操作)

# 2. 创建分区
> n      # 新建分区
> 1      # 分区号
>        # 默认起始扇区
> +512M  # 大小
>        # 默认类型 (Linux filesystem)

# 3. 保存退出
> w

# 探测新创建的分区
partprobe
```

gdisk 特点:

- 专为 GPT 分区设计
- 交互式操作, 界面类似 fdisk, 可替代fdisk
- 支持大容量磁盘 (>2TB)
- 自动分配唯一 GUID 给每个分区

优势:

- 对GPT支持最好
- 不需要计算起始和结束扇区位置
- w写入分区表后才执行分区, 可以提前确认

parted - rhel推荐的分区工具:

```
# 查看分区表(指定单位为MiB)
parted /dev/vdb unit MiB print

# 交互式创建分区
parted /dev/vdb
> mklabel gpt          # 创建 GPT 标签
> mkpart xfs 2048s 1000MiB # 创建1000MiB大小的新分区，需要计算起始和结束扇区位置
> quit #退出交互式

# 非交互式
parted /dev/vdb mkpart primary xfs 2048s 1001MiB
```

注意：

- parted 默认使用 10 进制单位(MB)
- 使用 MiB 表示二进制单位
- 2048s = 1MiB (起始位置)

5.2 创建文件系统

mkfs - 格式化分区：

```
# XFS 文件系统
mkfs.xfs /dev/vdb1

# ext4 文件系统
mkfs.ext4 /dev/vdb1
```

XFS vs EXT4 文件系统特性对比：

特性	XFS	EXT4
最大文件大小	8EiB (Exabytes)	16TiB
最大文件系统大小	16EiB	1EiB
扩容性	只能扩容，不能缩容	支持扩容和缩容
日志	内置日志，元数据日志	内置日志
配额	支持 project quota	支持 user/group quota

特性	XFS	EXT4
默认在 RHEL9	✔ 默认	可选

示例：

```
mkfs.xfs /dev/vdb1
```

SHELL

```
# 输出示例：
# meta-data=/dev/vdb1 isize=512 agcount=4, agsize=25600 blks
#          =                               sectsz=512   attr=2, projid32bit=1
#          =                               crc=1        finobt=1, sparse=1,
rmapbt=0
#          =                               reflink=1    bigtime=0
inobtcounting=1
# data      =                               bsize=4096   blocks=64000,
imaxpct=25
#          =                               sunit=0      swidth=0 blks
# naming    =version 2                     bsize=4096   ascii-ci=0
ftype=1
# log       =internal log                  bsize=4096   blocks=3840,
version=2
#          =                               sectsz=512   sunit=0 blks, lazy-
count=1
# realtime =none                          extsz=4096   blocks=0,
rtextents=0
```

第六部分：挂载文件系统

6.1 手动挂载

mount 命令：

```
# 创建挂载点
mkdir /mnt/data

# 挂载文件系统
mount /dev/vdb1 /mnt/data

# 查看挂载
mount | grep vdb1

# 输出示例:
# /dev/vdb1 on /mnt/data type xfs
(rw,relatime,seclabel,attr2,inode64,noquota)
```

按 UUID 挂载:

```
# 查看 UUID
lsblk -f /dev/vdb1

# 使用 UUID 挂载
mount UUID="363d9a26-df12-4967-b7fd-36c3a18345d6" /mnt/data
```

6.2 持久挂载

/etc/fstab 配置文件:

```
# /etc/fstab 字段
# 设备      挂载点  类型  选项      dump  fsck
UUID=xxx /mnt/data xfs defaults 0 0
```

字段说明:

字段	说明
第1字段	设备名或UUID
第2字段	挂载点目录
第3字段	文件系统类型
第4字段	挂载选项

字段	说明
第5字段	dump备份(0=不备份)
第6字段	fsck检查(0=不检查,1=根,2=其他)

添加持久挂载:

```
# 获取 UUID
blkid /dev/vdb1

# 编辑 fstab
vim /etc/fstab

UUID=363d9a26-df12-4967-b7fd-36c3a18345d6 /mnt/data xfs
defaults 0 0

# 测试挂载
mount -a
```

SHELL

⚠ 易错点: `/etc/fstab` 配置错误导致启动失败

fstab 字段说明:

```
UUID=xxx /mnt/data xfs defaults 0 0
```

```
| | | | |
| | | | | └─ fsck 检查顺序
| | | | └─ dump 备份
| | | └─ 挂载选项
| | └─ 文件系统类型
| └─ 挂载点目录
└─ 设备标识 (推荐使用 UUID)
```

常见错误:

- 设备名错误 (如 `/dev/vdb1` 写成 `/dev/vdb2`)
- UUID 复制错误 (多开终端导致 UUID 混淆)
- 挂载点目录不存在
- 文件系统类型错误 (`xfs` 写成 `ext4`)

- 选项中有拼写错误

安全步骤：

1. 编辑 fstab 后立即运行 `mount -a`
2. 检查是否有错误输出
3. 重启前确认所有挂载正常

`mount -a` 成功 = 无输出，失败 = 报错并指出问题

紧急恢复：

如果系统无法启动：

1. 重启，在 GRUB 界面按 `e` 编辑启动参数
2. 在 `linux16` 行末添加: `rd.break`
3. 按 `Ctrl+x` 启动
4. remount `/sysroot` 为 rw: `mount -o remount,rw /sysroot`
5. `vi /sysroot/etc/fstab` 修正错误
6. exit 重新启动

相关练习：RH134V9

练习名称	对应章节	说明
练习： ADDING PARTITIONS, FILE SYSTEMS, AND PERSISTENT MOUNTS	第07章 P220	添加分区、文件系统和持久化挂载

第七部分：交换空间

7.1 SWAP 基础

什么是 **SWAP**：

SWAP 是磁盘空间，用作虚拟内存的扩展

- 当 RAM 不足时使用
- 速度比 RAM 慢
- 不应长期依赖

SWAP 大小建议：

RAM 大小	SWAP 大小	休眠
≤ 2GB	2×RAM	3×RAM
2-8GB	=RAM	2×RAM
8-64GB	4GB	1.5×RAM
>64GB	4GB	不支持休眠

注意：

- SWAP 空间比 RAM 慢得多
- 不应长期依赖 SWAP
- 系统内存不足时才会使用 SWAP

7.3 SWAP 优先级详解

设置 SWAP 优先级：

SHELL

```
# 在 fstab 中添加 pri 选项
UUID=xxx swap swap defaults,pri=1 0 0

# 多个 SWAP 的优先级示例
UUID=aaa swap swap defaults,pri=1 0 0
UUID=bbb swap swap defaults,pri=2 0 0
UUID=ccc swap swap defaults,pri=3 0 0
```

优先级说明：

- 默认优先级：-2
- 值越高优先级越高 (1 > 0 > -1 > -2)
- 相同优先级时轮询使用 (round-robin)

⚠ 易错点：SWAP 优先级设置

SWAP 优先级 (pri) 规则：

规则
值越大
默认值
相同优先级
不同优先级

示例场景：

SHELL

```
# SSD SWAP (更快)
UUID=aaa swap swap defaults,pri=10 0 0

# HDD SWAP (较慢)
UUID=bbb swap swap defaults,pri=5 0 0
```

系统优先使用 SSD SWAP

验证命令：`swapon --show` 查看 PRIO 列确认优先级

查看 **SWAP** 状态：

SHELL

```
swapon --show

# 输出示例:
# NAME      TYPE      SIZE USED PRIO
# /dev/vdb1 partition 512M  0B  -2

# 使用 free 查看
free -h

# 输出示例:
#              total            used            free            shared
buff/cache    available
# Mem:        1.8Gi          150Mi          1.5Gi           10Mi
150Mi        1.5Gi
# Swap:       512Mi           0B           512Mi
```

SWAP 使用场景：

- 系统内存不足时临时扩展内存
 - 支持休眠功能 (需要足够的 SWAP 空间)
 - 避免系统内存耗尽导致的 OOM (Out of Memory)
-

7.2 创建 SWAP 分区

步骤1: 创建分区

```
gdisk /dev/vdb
> n
> p
> 1
> +512M
> w
```

SHELL

步骤2: 格式化 SWAP

```
mkswap /dev/vdb1
```

```
# 输出示例:
# Setting up swapspace version 1, size = 512 MiB
# UUID=617f0dc1-d334-4801-b49c-f6174c2665af
```

SHELL

步骤3: 激活 SWAP

```
# 激活 swap 分区
swapon /dev/vdb1

# 验证
swapon --show

# free 查看
free -h
```

SHELL

步骤4：持久化 SWAP

SHELL

```
# 编辑 fstab
vim /etc/fstab

UUID=617f0dc1-d334-4801-b49c-f6174c2665af swap swap defaults
0 0
```

相关练习：

练习名称	对应章节	说明
练习：MANAGING SWAP SPACE	第07章 P228	管理交换空间
总练习：MANAGING BASIC STORAGE	第07章 P232	基本存储管理综合练习

今日总结

命令清单

SELinux：

SHELL

getenforce	# 查看 SELinux 模式
setenforce 0/1	# 临时切换模式
semanage fcontext -a -t	# 添加上下文规则
restorecon -Rv	# 应用上下文
getsebool -a	# 列出布尔值
setsebool -P	# 永久设置布尔值

存储：

SHELL

lsblk	# 查看块设备
parted /dev/vdb print	# 查看分区表
mkfs.xfs /dev/vdb1	# 创建XFS文件系统
mount /dev/vdb1 /mnt	# 挂载文件系统
lsblk -f	# 查看UUID
blkid /dev/vdb1	# 查看UUID

SWAP:

```
mkswap /dev/vdb1      # 格式化SWAP
swapon /dev/vdb1      # 激活SWAP
swapon --show         # 查看SWAP
free -h               # 查看内存
```

SHELL

常见任务

1. 修复 SELinux 文件上下文

```
# 查看上下文
ls -Z /var/www/html/file

# 添加规则
semanage fcontext -a -t httpd_sys_content_t '/var/www/html/file'

# 应用上下文
restorecon -Rv /var/www/html
```

SHELL

2. 创建持久挂载

```
# 创建分区(parted为例)、格式化、挂载
parted /dev/vdb mkpart primary xfs 1MiB 512M
mkfs.xfs /dev/vdb1
mkdir /mnt/data
mount /dev/vdb1 /mnt/data

# 获取UUID
blkid /dev/vdb1

# 添加到 fstab
vim /etc/fstab
# UUID=xxx /mnt/data xfs defaults 0 0

# 测试
mount -a
```

SHELL

3. 创建 SWAP 分区

```
# 创建512M swap分区
parted /dev/vdb mkpart linux-swab 1MiB 513MiB
mkswap /dev/vdb1
swapon /dev/vdb1

# 添加到 fstab
vim /etc/fstab
# UUID=xxx swap swap defaults 0 0
```

4. 切换 SELinux 模式

```
# 临时切换
setenforce 0
setenforce 1

# 永久配置
vim /etc/selinux/config
# SELINUX=enforcing
```

命令速查表

命令	作用	常用选项
getenforce	查看SELinux模式	无
setenforce	切换模式	0, 1
semanage fcontext	管理上下文	-a, -t, -d
restorecon	应用上下文	-R, -v
ls -Z	查看上下文	无
parted	分区工具	print, mkpart
fdisk	分区工具	无
mkfs.xfs	创建XFS	无
mkfs.ext4	创建ext4	无
mount	挂载	无, -a
lsblk	查看块设备	-f
mkswap	格式化SWAP	无
swapon	激活SWAP	-a, --show

感谢参与!

下节课见! 🐧